

Design and Evaluation of a CNN-Based Voice Recognition System

Matuszewska Paula Staszewska Barbara Świercz Kamila Włodarczyk Wiktoria

January 27, 2026

1 Collecting data

We've decided to create the dataset by ourselves. All of the recordings we use in our project come from Youtube and the public videos that are shared there, which is legal as Youtube allows using their videos for educational purposes.

The recordings are stored as the .wav format as a mono audio with the same sample rate equal to 16000. We've chosen the .wav format as it does not compress the audio like other popular formats such as .mp3. Later on in the pre processing stage, which took place during segmentation of the recordings, we looked at things like normalization, silence removal, and noise removal.

1.1 Class Definition and Speaker Selection

The dataset is divided into 2 classes:

- **Class 0:** Consists of speakers who represent the general population that should not be accepted.
- **Class 1:** Consists of chosen speakers who have the access and their voice should be accepted.

We chose 5 speakers that will belong to class 1 and 20 speakers that belong to class 0. We have more speakers in class 0 so that it can represent a wider group of people which helps the model recognize the features of a speaker.

The speaker selection was done by each person working on this part separately. Of course we made sure no speaker was chosen twice. We focused on choosing public people, as they seemed a good choice since there are many recordings of them in different scenarios and times. When choosing the speakers we made sure some of them are older, some are different genders, and that some, especially for class 1, have very distinct and specific sounding voices.

For class 1: We focused on getting much more data per speaker so that the model trains to recognize their features more accurately. For each speaker we've collected 15 voice recordings that are around 90 seconds. We made sure that they are diverse so each one of them is from a different day, situation and environment. This gave us a total of around 1350 seconds of voice recordings per speaker. In total for class 1 we've collected around 112 minutes of voice recordings.

For class 0: We focused more on the variety of voices and to include different types of people, so that it would cover the general public. For each speaker we've collected 5 recordings, each one around 60 seconds. We made sure that they are diverse and that some of them have better and worse quality. This gave us a total of around 300 seconds of recordings per each speaker, which is much less than what we did for class 1. Additionally we included 3 recordings of random people speaking for a short amount of time, one or two sentences. In total these summed up to around 10 minutes of voices. Of course we made sure that no speaker from class 1 is in those recordings. In total for class 0 we've collected around 110 minutes of voice recordings.

All of this data in total gave us 5043 3-second-long spectrograms, the process of creating them will be explained later, which was enough for our model to have data to train on correctly. Later on combined with the augmented versions of this data we reach even better results.

2 Data Segmentation and Data Splitting

This stage converts the collected voice recordings into a structured set of fixed-duration audio excerpts and assigns them to **train** / **validation** / **test** partitions in a reproducible manner. The design follows the project’s emphasis on data cleaning (including removal of silent passages), choosing the excerpt length, and input normalization prior to spectrogram generation and CNN training.

2.1 Pre-segmentation Cleaning (Speech-Focused Waveform Construction)

Each recording is loaded as a **mono** waveform at a fixed sampling rate of **16 kHz**. To reduce variability caused by pauses and non-speech intervals, a two-step silence suppression procedure is applied:

- **Endpoint trimming:** leading and trailing low-energy regions are removed using an energy threshold ($TOP_DB = 35$) with a fixed analysis window ($FRAME_LENGTH = 2048$, $HOP_LENGTH = 256$).
- **Internal silence removal:** remaining low-energy intervals inside the trimmed signal are detected using `librosa.effects.split` with a stricter threshold ($INTERNAL_DB = 25$). The detected non-silent intervals are concatenated to form a “cleaned” waveform that primarily contains speech content.

This produces a speech-dominant signal that is then used consistently both for **split estimation** (duration measurement) and **chunk extraction**.

2.2 Segmentation into Fixed-Length Excerpts

Cleaned recordings are segmented into **non-overlapping 3-second** excerpts ($MAX_SEG = 3.0$). Segmentation proceeds sequentially with a fixed target length of $MAX_SEG * sample_rate$ samples. Any trailing remainder shorter than the target segment length is **discarded** (i.e., no padding is applied in the current segmentation loop).

2.3 Split Strategy: Duration-Based Allocation with Reproducible Shuffling

To construct train/validation/test partitions, the splitter estimates the **effective duration** of each original file after cleaning (i.e., after endpoint trimming and internal silence removal). Files are then randomly shuffled with a fixed seed ($SEED = 300$) to ensure reproducibility. Two duration-based allocators are implemented:

- **Speaker-disjoint split (used for Class 0):** cumulative cleaned duration is computed per speaker and speakers are assigned greedily to partitions (approx. 80% train / 10% validation / 10% test). This enforces that **no speaker identity occurs in more than one of train/validation/test**, eliminating speaker leakage across splits.
- **Recording-disjoint split (used for Class 1):** within each speaker directory, cleaned duration is computed per original file and files are assigned to partitions (approx. 80% train / 10% validation / 10% test). All segments extracted from a given recording inherit the recording’s assigned split, ensuring that **no single original file is divided across train/validation/test**.

A diagnostic routine recomputes and prints the realized duration ratios for the resulting partitions, providing an explicit check that the greedy allocation achieves the intended proportions.

2.4 Handling Augmented Recordings (Train-Only Inclusion)

Augmented recordings are detected via filename suffixes:

`_noise.wav, _chop.wav, _fast.wav, _slow.wav.`

They are excluded from split-ratio computations (their contribution to duration estimation is skipped). During segmentation, each augmented file is mapped back to its corresponding base recording name; the augmented variant is segmented **only if the base recording belongs to the training split**. This prevents augmented material from leaking into validation/test partitions.

2.5 Exporting Segmented Audio

For every retained 3-second excerpt, two versions are written:

- **Raw excerpt**
- **Peak-normalized excerpt**, obtained by scaling by the maximum absolute amplitude (if non-zero)

Both are stored as 16-bit PCM WAV files (PCM_16) into a directory structure that encodes the partition and class, enabling downstream spectrogram conversion and model training to operate directly on `train_data`, `validation_data`, `test_data`). Augmented segments are additionally identifiable through a suffix appended to the file prefix (e.g., `_noise`, `_chop`, `_fast`, `_slow`).

3 Log-Mel Spectrogram Generation

All audio signals are processed at a sampling rate of 16 kHz and converted to mono. Each fixed-length audio excerpt is represented as a log-Mel spectrogram serving as the CNN input.

Given a discrete-time signal $x[n]$, the short-time Fourier transform (STFT) is computed using a Hann window with a duration of 25 ms and a hop size of 10 ms ($n_{\text{fft}} = 400$, hop length 160 samples). The squared magnitude of the STFT coefficients yields a power spectrogram, which is projected onto the Mel scale using a triangular filterbank with $F = 80$ Mel bands spanning 20–7600 Hz.

Let $S(f, t)$ denote the resulting Mel-scaled power spectrogram. Logarithmic compression is applied as

$$X(f, t) = 10 \log_{10} \left(\frac{S(f, t)}{\max_{f, t} S(f, t)} \right),$$

mapping the maximum spectral energy of each sample to 0 dB.

To obtain a fixed input shape, spectrograms are center-trimmed or zero-padded along the temporal axis. For 1-second excerpts this yields approximately $T \approx 100$ time frames, while 3-second excerpts contain $T \approx 300$ frames, with the frequency dimension fixed at $F = 80$.

3.1 Effect of Segment Length: 1 s vs 3 s

This experiment investigates the influence of excerpt duration on model generalization while keeping the training setup unchanged. The only variable is the segment length used prior to feature extraction.

Segment length	Test Acc.	Test Loss	Macro F1	Weighted F1
1 s (raw)	0.5110	0.9541	0.5034	0.5346
3 s (raw)	0.8731	0.4373	0.8721	0.8739

Table 1: Performance comparison for different excerpt lengths (raw log-Mel spectrograms).

As shown in Table 1, 3-second excerpts substantially outperform 1-second excerpts on the test set. While shorter segments increase the number of training samples, they provide limited temporal context and fail to capture sufficient speaker-specific information. Based on this comparison, all subsequent experiments use 3-second excerpts.

3.2 Input Normalization (3 s Segments)

To study the effect of input scaling, several normalization strategies were applied directly to the log-Mel spectrograms computed from 3-second excerpts. Let $X \in \mathbb{R}^{F \times T}$ denote a log-Mel spectrogram.

No normalization. As a baseline, spectrograms are used directly after log-scale conversion.

Global z -score normalization. A single mean μ_{train} and standard deviation σ_{train} are computed over all training spectrogram values and applied to all splits:

$$X_{\text{norm}} = \frac{X - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

Per-clip z -score normalization (CMVN). Each spectrogram is normalized independently:

$$X_{\text{norm}} = \frac{X - \mu_X}{\sigma_X},$$

where μ_X and σ_X are computed over all time–frequency bins of a single sample.

Per-frequency global normalization. Normalization parameters are computed separately for each Mel band:

$$X_{\text{norm}}(f, t) = \frac{X(f, t) - \mu_f}{\sigma_f}.$$

Min–max scaling. Each spectrogram is rescaled to a fixed range:

$$X_{\text{norm}} = \frac{X - \min(X)}{\max(X) - \min(X)}.$$

Robust scaling. Robust normalization clips extreme values using percentiles:

$$X_{\text{norm}} = \frac{\text{clip}(X, P_5, P_{95}) - P_5}{P_{95} - P_5}.$$

Normalization	Test Acc.	Macro F1	Weighted F1
None (raw)	0.8731	0.8721	0.8739
Global z -score	0.7864	0.7831	0.7874
Min–max	0.7214	0.7156	0.7221
Robust (percentile)	0.7136	0.7049	0.7131
CMVN (per-clip)	0.6966	0.6803	0.6919
Per-frequency global	0.6966	0.6842	0.6943

Table 2: Normalization comparison for 3-second excerpts (test set).

Overall, the best performance is achieved without additional normalization beyond the log-Mel representation. Global z -score normalization provides the strongest alternative, while per-clip and per-frequency normalization reduce performance, likely due to suppression of amplitude-related cues relevant for speaker discrimination.

4 Spectrogram Dataset Preparation for Model Input

After converting the audio excerpts into mel-spectrograms and storing them as NumPy arrays (`.npy`), the data are loaded into the training/evaluation via a custom PyTorch Dataset class, `NpySpectrogramDataset`. This component is responsible for:

- assigning class labels,
- optionally including a controlled amount of augmented samples, and
- enforcing shape consistency before returning tensors in the format expected by a CNN.

4.1 Dataset Organization and Label Mapping

The dataset loader expects spectrograms to be stored in a structured folder layout:

```
root/<class_name>/<speaker_id>/*.npz.
```

Class labels are inferred directly from the top-level class directory and converted to numeric targets using a configurable mapping (`class_to_label`, default to `class_0` $\rightarrow$ 0 and `class_1` $\rightarrow$ 1).). During initialization, the loader iterates through each class folder and its speaker subdirectories, collects all `.npz` spectrogram files, and stores each sample as a (`file_path`, `label`) pair. This keeps the labeling procedure explicit and consistent, while still preserving the speaker-based organization of the dataset.

4.2 Controlled Inclusion of Augmented Spectrograms

The loader distinguishes original from augmented samples by inspecting the filename. Files containing any of the substring `_noise_`, `_chop_`, `_fast_`, or `_slow_` are treated as augmented variants. Augmented samples are only included if the corresponding augmentation type is enabled (via `use_noise`, `use_chop`, `use_fast`, `use_slow`).

Additionally, the parameter `aug_ratio` provides a simple mechanism to control how many augmented samples are incorporated:

- all eligible augmented samples are first collected,
- then a reproducible random subset of size

$$k = \text{round}(\text{len}(\text{aug_samples}) \cdot \text{aug_ratio})$$

is sampled using a fixed seed.

This allows experiments that systematically vary augmentation strength while keeping the selection deterministic across runs.

4.3 Deterministic Sample List Construction

The final sample list is created as:

```
self.samples = original_samples + aug_samples
```

and then sorted. This sorting step ensures a stable ordering independent of OS directory listing behavior, which is useful for reproducibility and debugging.

4.4 Shape Validation and Tensor Formatting

Each spectrogram is loaded with `np.load(path)` and expected to represent a 2D time–frequency map. If the loaded array has extra singleton dimensions, they are removed via `np.squeeze`.

Two explicit integrity checks enforce consistent model input dimensions:

- the number of mel bins must equal `mel_bins` (default: 80),
- the time dimension must equal `expected_width`.

If either check fails, the dataset raises an error immediately. This “fail fast” behavior prevents silent shape mismatches that would otherwise appear later as model runtime errors.

Finally, the array is converted to a PyTorch tensor of type `float32` and reshaped to include an explicit channel dimension:

- input: `[mel_bins, width]`
- output: `[1, mel_bins, width]`

4.5 Output Interface

The dataset returns tuples of the form $(\mathbf{x}, y)$, where:

- x is a single-channel spectrogram tensor ready for CNN ingestion,
- y is the integer class label.

In short, `NpySpectrogramDataset` acts as a bridge between saved spectrograms (`.npy`) and a strictly validated, reproducible tensor representation suitable for training and evaluation.

5 Data Augmentation

To increase our database we augmented our original recordings. For augmentation we picked 4 different types of it:

- adding noise,
- speeding up the recording,
- slowing down the recording,
- chopping and shuffling chunks of the recording.

These augmentations were done on all data in our previously collected database.

It's important to note here that we did not stack the augmentations, that is, once an augmented recording was created, it was not used for any other augmentations. This was to ensure that no unrealistic or unrecognizable data was created.

The augmentation was done on full recordings (around 60/90 seconds depending on the class they are from).

We also did not perform any augmentations that would mix any recordings from different speakers or classes. As a matter of fact, no augmentation performed by us took data outside of the specific recording it was augmenting at that moment. This is important to mention because any mixing of recordings or speakers could worsen the results of the model training as voices could become indistinguishable.

Before any augmentation we did basic preprocessing of the recording like silence removal and setting the sample rate to 16000 Hz. We feared that if we did not preprocess it before the augmentation, it could be hard to preprocess the data after it.

To understand the benefit of each of the augmentations it's best to explain them:

Adding Noise The augmentation added a noise filter over the voice recording. Random noise is added with a randomly selected signal-to-noise ratio. This allows us to control the noise compared to the speech signal. Because the ratio is random, this can represent the recordings made in different acoustic environments for a speaker. This is the most important augmentation, and was crucial for a model to train to recognize the voice in a worse environment better.

Chopping and Shuffling The augmentation first divided the recording into many small segments which were then randomly reordered. The chopped segments are smaller than the later on segmentation to ensure that the chopping affects the spectrograms later on.

The outcome is a recording of random words or parts of a word made by a speaker, usually the random words do not make any coherent sense together. This is done to teach the model to recognize not specific phrases or words, but the voice of the speaker itself. Thanks to this we hope that the model will learn the speaker's voice well enough to recognize words that were not said by the speaker in the original recordings in the database.

Speeding Up The augmentation shrinks the audio of a recording, resulting in an audio that plays faster while preserving the original pitch. In this augmentation it is important to limit the speed increase to ensure that the audio does not become less than the minimum seconds required to make a spectrogram later on, as well as to not let the audio become too unnatural or distorted.

This augmentation should benefit in the model recognizing the speaker even if they speak faster than in the recording in the dataset.

Slowing Down This augmentation stretches out the audio of the recording, resulting in a slowed down audio. This helps represent the real speech where a speaker talks significantly slower than usual. In result we hope this helps the model to not rely on the speed of the recording but rather the features of the voice itself.

5.1 Summary

To summarize, the augmentations not only helped the overfitting problem as they gave us 4x of the original data to work with, but they also tackled the issues of the model depending on the clarity of speech, or the words themselves. Later on our focus is to find the perfect ratio of these augmentations in the dataset compared to the original data, looking at which augmentations will have the best results when trained on to later on use in training the models.

6 Model-Introduction

The training procedure is implemented in a standalone script that handles reproducibility, data loading, training, validation, checkpointing, and final testing in a consistent way across experiments.

At the start of each run, random seeds are set and deterministic CUDA behavior is enabled to improve repeatability. The script then builds `DataLoaders` for the training, validation, and test splits using the previously mentioned custom `NpySpectrogramDataset` class, ensuring that class-to-label mappings remain consistent across splits.

During training, the model is optimized over multiple epochs using a chosen optimizer and loss function (varied across experiments). After each epoch, the script computes training loss/accuracy and evaluates the model on the validation set. The best-performing checkpoint is automatically saved (based on a minimum validation loss), allowing a fair comparison between different configurations.

After training finishes, the script reloads the best checkpoint and performs a single final evaluation on the test set. In addition to loss and accuracy, it computes macro- and weighted-F1 scores to provide a more informative metric summary. Training curves and metrics are logged to TensorBoard, and all recorded metrics are saved for later analysis and comparison across runs.

7 Bad Models (Simple CNN)

7.1 SimpleCNN (Baseline Model)

As an initial baseline, we implemented a compact convolutional neural network (*SimpleCNN*) to classify the mel-spectrogram inputs into the two target classes. The main motivation for starting with a simple architecture was to establish a reliable reference point and to validate the end-to-end pipeline (spectrogram generation $\rightarrow$ dataset loading $\rightarrow$ training loop $\rightarrow$ evaluation) before moving on to more complex models.

SimpleCNN takes fixed-size mel-spectrograms of shape $1 \times 80 \times 300$ (single channel, 80 mel bins, 300 time frames) as input. Feature extraction is performed by two convolutional blocks. The first block applies a 5×5 convolution mapping from 1 to 32 channels, followed by a ReLU activation and 2×2 max-pooling. The second block repeats the same pattern with a 5×5 convolution expanding from 32 to 64 channels, again followed by ReLU and 2×2 max-pooling. The pooled feature maps are then flattened and passed to a fully connected classifier consisting of two hidden layers (120 and 84 units, with ReLU activations) and a final linear layer producing two output logits.

7.2 SimpleCNN experiments and results overview

The statistics below provide an overview of **SimpleCNN performance across different experimental configurations**. For each configuration (i.e., each model type/architectural variant), we trained **multiple runs with different hyperparameters** (e.g., learning rate, weight decay, optimizer choice, regularization strength). In the table, we report **only the best-performing hyperparameter setting for each configuration**, selected according to validation performance, and then evaluated on the test set.

7.3 Experimental protocol

We followed an iterative development and selection procedure:

- Start from the baseline SimpleCNN architecture.
- Introduce one architectural or training modification at a time (e.g., batch normalization, dropout, pooling, additional convolutional depth, augmentation).
- For each configuration, run several trainings with different hyperparameters, and keep the best run based on validation performance.
- If a modification improved validation performance, it was retained as part of the architecture in subsequent runs; if it did not, it was removed. In this way, later configurations are built on the strongest elements discovered earlier rather than restarting from scratch.
- Select the best checkpoint for each configuration based on validation performance (as reflected by the reported “Epoch” and validation metrics), and compute final test metrics using that checkpoint.

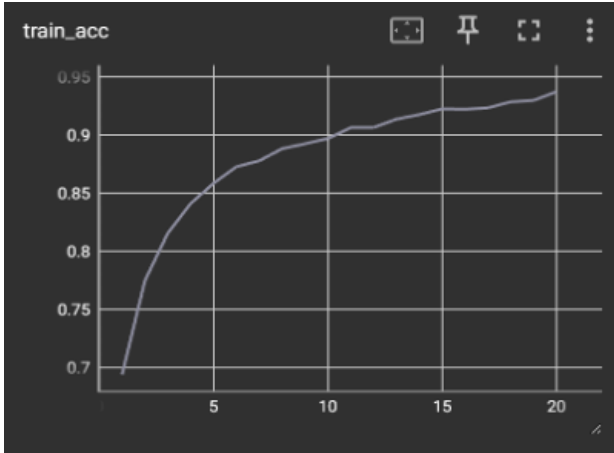
Table 3: imple CNN results

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 (Macro)	Test F1 (Weighted)
1	2	0.566917	0.687057	0.478328	0.740408	0.471950	0.462606
2	1	0.593985	0.731639	0.473684	0.792535	0.464632	0.453425
3	1	0.619549	0.691213	0.441176	0.759051	0.414859	0.394882
4	1	0.598496	0.677279	0.465944	0.726967	0.463568	0.457819
5	7	0.575940	0.684112	0.518576	0.718846	0.517965	0.515202
6	2	0.598496	0.741832	0.552632	0.799929	0.523782	0.542652
7	19	0.624060	0.706400	0.568111	0.835076	0.557724	0.546812
8	1	0.633083	0.670345	0.421053	0.856504	0.419629	0.415000
9	1	0.650092	0.651591	0.581395	0.772511	0.560504	0.561061

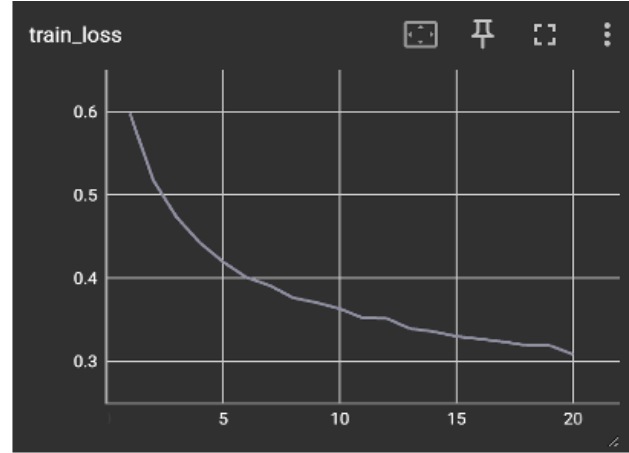
Description of model configurations

- **Model 1: Baseline + SGD** — baseline architecture trained with SGD ($\text{lr} = 0.01$, $\text{wd} = 5e-4$).
- **Model 2: Adam optimizer** — Adam ($\text{lr} = 3e-4$, $\text{wd} = 1e-3$).
- **Model 3: Convolutional dropout** — dropout in the convolutional block ($p = 0.3$).
- **Model 4: Fully connected dropout** — dropout applied in the fully connected layers ($p = 0.5$).
- **Model 5: Convolutional batch normalization** — batch normalization added to convolutional layers.
- **Model 6: Fully connected batch normalization** — batch normalization added to fully connected layers.

- **Model 7: Adaptive global pooling** — introduce adaptive global pooling to reduce sensitivity to input width and improve feature aggregation.
- **Model 8: Deeper CNN** — add a third convolutional layer to increase capacity.
- **Model 9: Full augmentation on best prior model** — apply an augmentation pipeline on top of the best previous configuration (Model 7).



(a) Model 9 - Train Accuracy



(b) Model 9 - Train Loss

7.4 Motivation for Moving Beyond SimpleCNN: ResNet-18

Because the SimpleCNN family remained prone to overfitting and achieved only moderate test performance despite systematic tuning, we next adopt **ResNet18** as a stronger baseline for spectrogram classification. ResNet18 is well suited to this setting because it provides a deeper and more expressive feature extractor while remaining stable to train due to residual connections. These skip connections improve gradient flow and help prevent performance degradation as depth increases.

We decided to implement a **custom ResNet18-style architecture**. Rather than using the torchvision model directly end-to-end, we **reimplemented the ResNet18 building blocks** ourselves to keep full control over the network structure and to adapt it cleanly to **single-channel spectrograms**.

8 ResNet18

8.1 Architecture Design

The model follows the standard ResNet18 pattern:

- **Residual BasicBlock:** each block contains two 3×3 convolution layers, each followed by batch normalization, with a ReLU nonlinearity. A **skip connection** adds the input to the block output.
- **Downsampling when needed:** When the spatial resolution changes or the number of channels increases, a 1×1 **convolution** + **batch normalization** is applied to the input path to match dimensions before addition.
- **Four residual stages:** The network is composed of four residual stages with channel sizes **64, 128, 256, 512**, each built from **2 residual blocks**, matching ResNet18 depth.

8.2 Input Adaptation for Spectrograms

Unlike ImageNet images (3-channel RGB), spectrograms are **single-channel**. Therefore:

- The first convolution is defined as `Conv2d(1  $\rightarrow$  64)` with a 7×7 kernel and stride 2.

- The network then applies the standard ResNet stem: batch normalization $\rightarrow$ ReLU $\rightarrow$ max pooling.

8.3 Classification Head

After the residual stages, the model uses:

- **Adaptive average pooling** to obtain a fixed-size representation independent of the exact input dimensions.
- A final **fully connected layer** that outputs logits for **two classes**, consistent with the binary classification setting.

9 Not-Pretrained Models with Augmentation

To identify the data augmentation strategy that yields the best performance for a not-pretrained ResNet18, a series of controlled experiments was conducted by systematically varying the augmentation configuration used during training. Each experiment was defined by five parameters:

- **AUG_RATIO**,

and four binary flags controlling the use of specific augmentation types:

- **USE_NOISE**,
- **USE_CHOP**,
- **USE_FAST**,
- **USE_SLOW**.

An augmentation ratio of 1.0 indicates that augmentations were applied to 100% of the training spectrograms, while 0.0 corresponds to training on original data only. The boolean flags specify whether a given augmentation type is enabled in the training pipeline.

In the first stage of experiments, all 16 possible combinations of the four augmentation options were evaluated using a fixed augmentation ratio of **AUG_RATIO = 1.0**. This allowed for a direct comparison of individual and combined augmentation effects. The results of these experiments are summarized in Table.

Table 4: Not-pretrained ResNet-18 with **AUG_RATIO = 1.0**: comparison of augmentation combinations. Abbreviations: N — Noise, C — Chop & shuffle, F — Fast, S — Slow.

Aug	Train Acc	Val Acc	Test Acc	Val F1 (Macro)	Val F1 (Weighted)	Test F1 (Macro)	Test F1 (Weighted)
0.0	0.9911	0.6059	0.7049	0.7499	0.7518	0.6949	0.6952
N	0.9848	0.7109	0.6817	0.7389	0.7417	0.6758	0.6760
C	0.9919	0.7882	0.7369	0.7992	0.7987	0.7369	0.7369
F	0.9940	0.7403	0.6860	0.7982	0.7996	0.6775	0.6778
S	0.9896	0.7551	0.7035	0.7897	0.7905	0.6988	0.6990
NC	0.9882	0.7845	0.6919	0.7797	0.7822	0.6801	0.6804
NF	0.9849	0.6427	0.6788	0.7239	0.7260	0.6685	0.6689
NS	0.9917	0.7311	0.6570	0.8028	0.8047	0.6439	0.6443
CF	0.9919	0.6869	0.7078	0.8209	0.8218	0.7061	0.7062
CS	0.9890	0.7090	0.7006	0.7734	0.7729	0.6991	0.6992
FS	0.9901	0.7624	0.6730	0.8068	0.8085	0.6614	0.6618
NCF	0.9913	0.7440	0.6701	0.8301	0.8310	0.6659	0.6661
NCS	0.9888	0.7311	0.6672	0.7923	0.7955	0.6550	0.6554
NFS	0.9928	0.7864	0.7122	0.7897	0.7889	0.7112	0.7113
CFS	0.9928	0.8066	0.7137	0.8411	0.8430	0.7051	0.7054
NCFS	0.9910	0.7127	0.6875	0.8144	0.8160	0.6827	0.6829

Based on these results, the chopping alone (C) achieved the highest test accuracy. Including additive noise, speeding up, or slowing down the recording did not improve performance for the not-pretrained model.

In the second stage, the selected C configuration was further analyzed by varying the augmentation ratio. The model was trained using **AUG_RATIO** values of 1.0, 0.8, 0.5, 0.3, and 0.2, corresponding to progressively smaller proportions of augmented samples in the training set. The comparative results for different ratios are reported in Table.

Table 5: Not-pretrained ResNet-18 with chopping augmentation (C): effect of **AUG_RATIO**.

Aug Ratio	Train Acc	Val Acc	Test Acc	Val F1 (Macro)	Val F1 (Weighted)	Test F1 (Macro)	Test F1 (Weighted)
1.0	0.9919	0.7882	0.7369	0.7992	0.7987	0.7369	0.7369
0.8	0.9868	0.7090	0.6788	0.7801	0.7850	0.6592	0.6597
0.5	0.9902	0.7790	0.7078	0.7941	0.7958	0.7041	0.7043
0.3	0.9885	0.6906	0.6875	0.7659	0.7680	0.6803	0.6806
0.2	0.9616	0.6685	0.6453	0.8172	0.8181	0.6435	0.6436

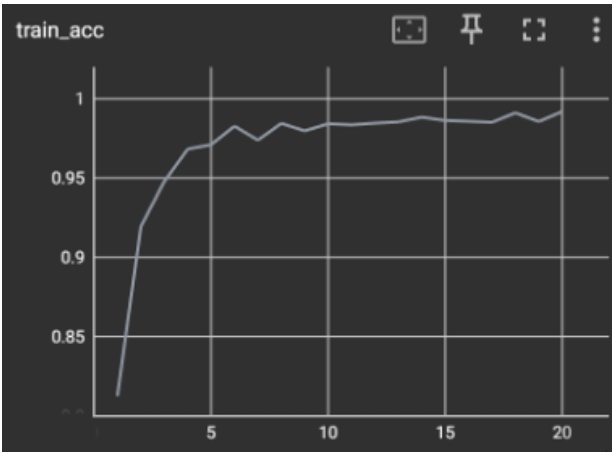
The highest performance was obtained with **AUG_RATIO** = 1.0. Therefore, for the subsequent model preparation, we will apply only the **chopping augmentation (C)** with an augmentation ratio of **1.0**, as this configuration provides the most effective training input for the not-pretrained ResNet18.

10 Model-Centered Experiments (ResNet Family)

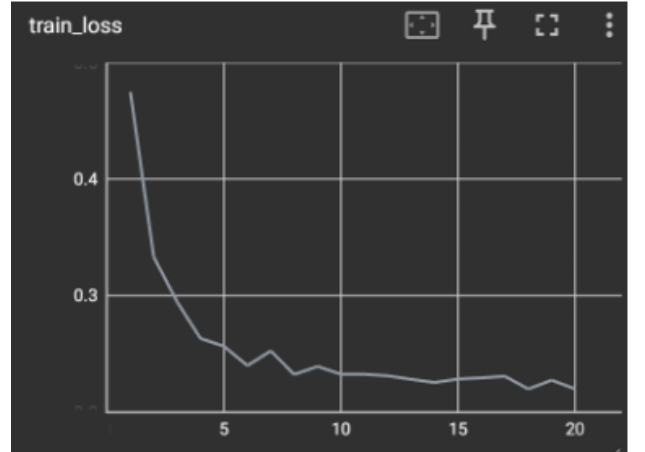
After identifying the most effective data augmentation setup, we shifted the focus from the input pipeline to the **model itself**. In this stage, we evaluated how different **architectural and structural modifications** influence performance, using the same training and evaluation protocol to keep comparisons fair. Concretely, we tested changes such as network depth/width, regularization mechanisms, and internal block design choices (e.g., skip connections, normalization and activation functions), and assessed whether each modification improved generalization on the validation and test sets.

10.1 Impact of Optimizer, Learning-Rate Scheduling, and Weight Decay

After establishing a baseline training setup, we systematically evaluated how **optimization choices** influence performance. In particular, we varied the optimizer family (**Adam**, **AdamW**, **SGD**), the learning rate and weight decay, and the use of common learning-rate schedulers (**StepLR**, **CosineAnnealing**, **ReduceLROnPlateau**). For each configuration, we trained the same architecture and selected the **best checkpoint based on the lowest validation loss observed across epochs**. The corresponding validation and test metrics of these best checkpoints are summarized in the table below.



(a) Model 1 - Train Accuracy



(b) Model 1 - Train Loss

Table 6: ResNet-18 optimizer, learning-rate scheduling, weight-decay

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
1	17	0.799263	0.501762	0.736919	0.597218	0.736891	0.736876
2	9	0.736648	0.544548	0.616279	0.795724	0.613137	0.613340
3	17	0.841621	0.443489	0.709302	0.651417	0.701096	0.701384
4	14	0.854512	0.466914	0.712209	0.750642	0.707854	0.708062
5	10	0.852670	0.455274	0.686047	0.777307	0.680330	0.680579
6	9	0.802947	0.507852	0.729651	0.745198	0.724729	0.724943
7	19	0.826888	0.511706	0.725291	0.729250	0.721620	0.721806
8	8	0.848987	0.447598	0.667151	0.800633	0.662704	0.662929
9	8	0.808471	0.552627	0.707849	0.746357	0.700327	0.700603
10	17	0.841621	0.481017	0.655523	0.880179	0.654910	0.654995
11	8	0.709024	0.583997	0.610465	0.798546	0.610136	0.610070
12	20	0.825046	0.476158	0.713663	0.741230	0.710031	0.710219
13	11	0.841621	0.466628	0.691860	0.749683	0.688448	0.688638
14	19	0.815838	0.500374	0.713663	0.727478	0.710581	0.710754
15	8	0.817680	0.497391	0.693314	0.745595	0.691341	0.691485
16	20	0.854512	0.447323	0.665698	0.857244	0.654474	0.654836
17	10	0.845304	0.450785	0.691860	0.760089	0.689655	0.689807

Description of Model Configurations To isolate the effect of optimization choices, we kept the network architecture and data pipeline fixed and varied the **optimizer**, **learning rate**, **weight decay**, and (when applicable) the **learning-rate scheduler**. Each numbered model below corresponds to one configuration in Table 6, where the reported metrics come from the checkpoint with the **lowest validation loss across epochs**.

Adam (Models 1–7).

- **Model 1 (baseline):** Adam, learning rate 3×10^{-4} and weight decay 1×10^{-3} .
- **Model 2:** Adam, learning rate 1×10^{-3} and weight decay 1×10^{-3} .
- **Model 3:** Adam, learning rate 3×10^{-4} and weight decay 3×10^{-3} .
- **Model 4:** Adam (baseline configuration) + StepLR (**step_size**=7, **gamma**=0.1).
- **Model 5:** Adam (baseline configuration) + CosineAnnealingLR (**T_max**=num_epochs, **eta_min**=1e-5).
- **Model 6:** Adam (baseline configuration) + CosineAnnealingLR (**T_max**=num_epochs/2).
- **Model 7:** Adam (baseline configuration) + ReduceLROnPlateau (**factor**=0.5, **patience**=2).

AdamW (Models 8–10).

- **Model 8:** AdamW, learning rate 3×10^{-4} and weight decay 1×10^{-3} (no scheduler).
- **Model 9:** AdamW, learning rate 3×10^{-4} and weight decay 3×10^{-3} .
- **Model 10:** AdamW, learning rate 1×10^{-3} and weight decay 1×10^{-4} .

SGD with momentum (Models 11–17).

- **Model 11:** SGD, learning rate 0.1, momentum 0.9 and weight decay 1×10^{-3} .
- **Model 12:** SGD, learning rate 0.01, momentum 0.9 and weight decay 1×10^{-3} .
- **Model 13:** SGD, learning rate 0.01, momentum 0.9 and weight decay 5×10^{-4} .

- **Model 14:** SGD (as in Model 12) + StepLR (`step_size=7`, `gamma=0.1`).
- **Model 15:** SGD (as in Model 12) + CosineAnnealingLR (`T_max=num_epochs`, `eta_min=1e-5`).
- **Model 16:** SGD (as in Model 12) + CosineAnnealingLR (`T_max=num_epochs/2`).
- **Model 17:** SGD (as in Model 12) + ReduceLROnPlateau (`factor=0.5`, `patience=2`).

Observations These experiments did not produce a single configuration that consistently outperformed the others across all metrics. The checkpoint with the **lowest validation loss** was obtained with **Adam and higher weight decay** (Model 3), but this did not yield the best test results. In contrast, the strongest **test accuracy and F1** were achieved by the **baseline Adam setup** (Model 1), even though its validation loss was not the lowest overall. Scheduler variants (StepLR, CosineAnnealing, ReduceLROnPlateau) and AdamW produced mixed outcomes—sometimes improving validation metrics, but without a reliable improvement on the test set. SGD configurations were generally less competitive in test performance under the explored settings.

Conclusions Overall, the main takeaway is that optimizer/scheduler changes affected the **validation–test trade-off**, and selecting solely by validation loss did not always align with the best test performance. For subsequent runs we therefore kept **Adam as the default** and evaluated further changes using both validation loss and test-set metrics.

10.2 Impact of Dropout

Impact of dropout After exploring optimizer and scheduler choices, we evaluated the effect of **dropout regularization** on model performance. All runs in Table 7 use the same base training setup as the previously selected best configuration (Model 1) and differ only in the **dropout placement** (fully connected vs. convolutional features) and the **dropout probability** p . As before, for each configuration we report the checkpoint with the **lowest validation loss** across epochs and its corresponding test-set performance.

Table 7: ResNet-18 dropout experiments (best checkpoint selected by lowest validation loss).

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
18	11	0.762431	0.584287	0.675872	0.744958	0.673937	0.674083
19	10	0.801105	0.515319	0.654070	0.819583	0.647489	0.647769
20	19	0.815838	0.468982	0.704942	0.699186	0.700584	0.700794
21	8	0.801105	0.487147	0.668605	0.718630	0.654713	0.655115
22	17	0.786372	0.550531	0.680233	0.736312	0.676692	0.676888
23	12	0.793738	0.508776	0.697674	0.722078	0.695199	0.695359
24	15	0.828729	0.464835	0.703488	0.722231	0.697089	0.697345
25	3	0.812155	0.478374	0.668605	0.737077	0.668535	0.668563
26	13	0.819521	0.495707	0.722384	0.696779	0.710570	0.710910
27	3	0.790055	0.508834	0.645349	0.784808	0.644481	0.644583

Three dropout strategies were compared:

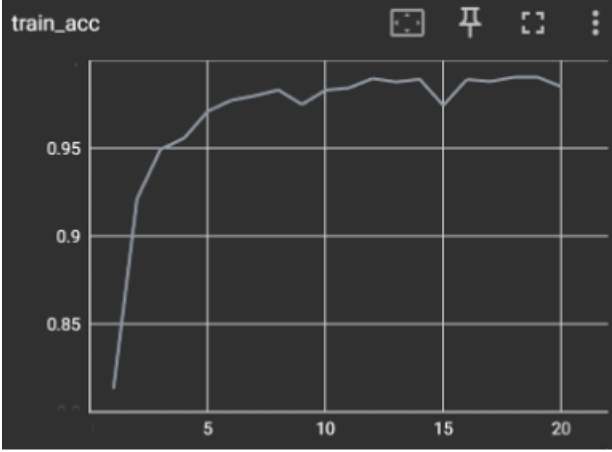
1. Dropout in the classifier (fully connected) layers:

- **Model 18:** FC dropout, $p = 0.5$.
- **Model 19:** FC dropout, $p = 0.3$.

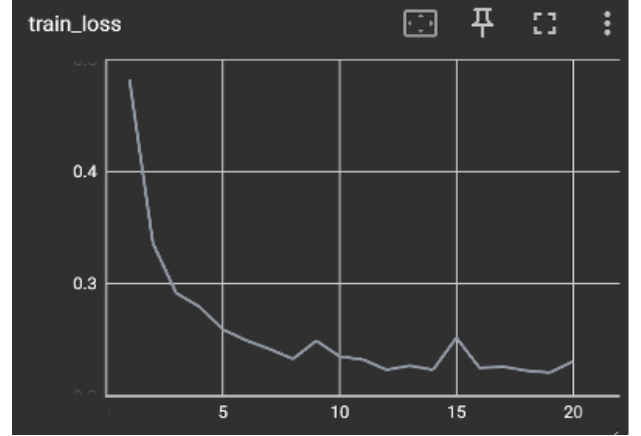
2. Dropout applied to convolutional feature maps:

- **Models 20–23:** convolutional dropout with $p \in \{0.10, 0.05, 0.20, 0.15\}$, respectively.

- **Models 24–27:** convolutional dropout applied after ResNet layer4 with $p \in \{0.10, 0.105, 0.15, 0.20\}$, respectively.



(a) Model 26 - Train Accuracy



(b) Model 26 - Train Loss

Observations Overall, dropout produced **mixed results**. Applying dropout only in the FC layers (Models 18–19 $p = 0.5$ and $p = 0.3$) led to weaker validation and test performance, suggesting that this form of regularization was not beneficial in this setting. Convolutional dropout variants (Models 20–27) achieved slightly better results than FC dropout in some cases, but the outcomes still varied depending on both placement and probability. The best dropout-based configuration in this block (Model 26) achieved the highest test accuracy among the dropout runs (≈ 0.722), but it remained below the baseline Model 1 test accuracy (≈ 0.737).

Conclusions Summarizing, In our setup dropout did not provide a consistent improvement over the baseline; at best it narrowed the gap while sometimes harming performance. For subsequent experiments, dropout was therefore treated as an optional regularization tool rather than a default component,

10.3 Impact of Different Layer Widths and Depths

10.4 ResNet-34

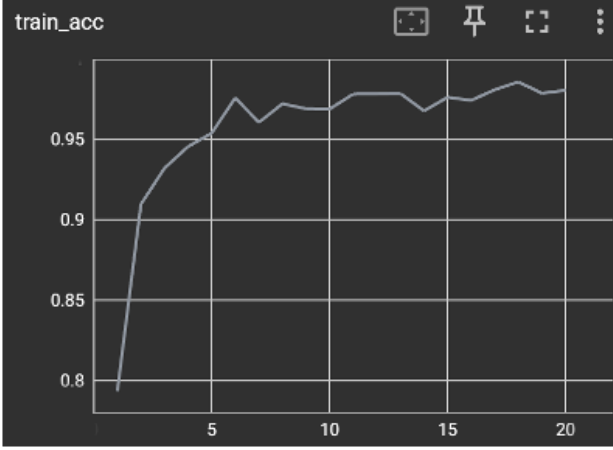
To evaluate how the **capacity** of a **deeper** residual network affects performance, we implemented and trained a ResNet-34 variant adapted to single-channel mel-spectrograms. The architecture follows the standard **ResNet-34 layout**. It begins with a 7×7 convolution (stride 2) followed by batch normalization, ReLU, and 3×3 max-pooling. Feature extraction then proceeds through four residual stages:

- Stage 1 (**layer1**): 3 residual blocks,
- Stage 2 (**layer2**): 4 residual blocks, first block uses stride 2,
- Stage 3 (**layer3**): 6 residual blocks first block uses stride 2,
- Stage 4 (**layer4**): 3 residual blocks first block uses stride 2.

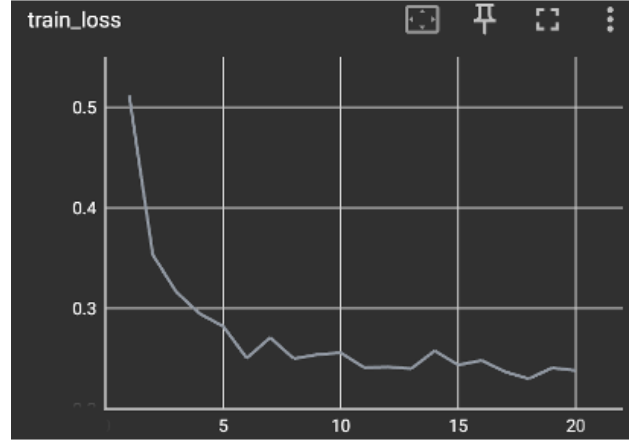
Each residual block contains two 3×3 convolutions with batch normalization and ReLU, and includes an identity skip connection. When spatial resolution or channel dimensionality changes, a 1×1 downsampling projection is applied to the skip path to enable residual addition. The network ends with global average pooling (`AdaptiveAvgPool2d((1,1))`) and a classifier producing **two logits**.

Table 8: ResNet-34

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
28	17	0.834254	0.459539	0.667151	0.771714	0.657584	0.657917
29	17	0.832413	0.501011	0.678779	0.872268	0.670817	0.671115
30	13	0.753223	0.529719	0.710756	0.601958	0.705605	0.705832



(a) Model 30 - Train Accuracy



(b) Model 30 - Train Loss

Compared configurations (Table 8). We compared three channel-width settings while keeping the block structure (3–4–6–3) fixed:

- **Model 28 (standard ResNet-34):** channel widths 64–128–256–512.
- **Model 29 (narrow ResNet-34):** channel widths 32–64–128–256.
- **Model 30 (wide ResNet-34):** channel widths 96–192–384–768.

Observations The **narrow** configuration (Model 29) achieved similar validation accuracy to the standard model but did not improve test performance (Test Acc ≈ 0.679). The **standard** configuration (Model 28) obtained slightly stronger validation accuracy (Val Acc ≈ 0.834) but showed weaker test generalization (Test Acc ≈ 0.667). The **wide** variant (Model 30) reached the best test performance among these three non-pretrained ResNet34 models (Test Acc ≈ 0.711), suggesting that increased channel capacity helped the network generalize better on the test set in this setup.

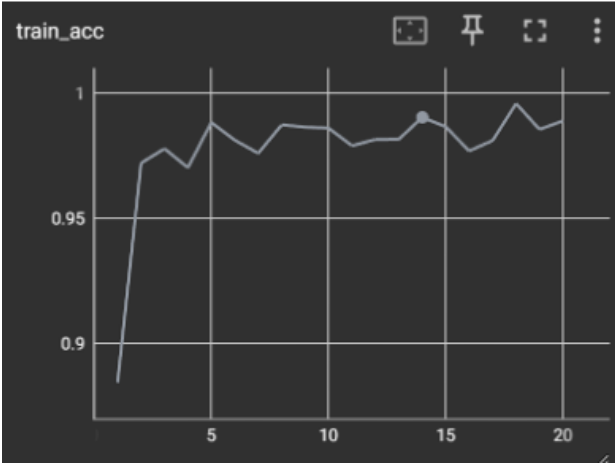
Conclusion Within ResNet34, changing only the channel widths affected the balance between validation and test results. In this experiment, increasing width improved test-set performance relative to the standard and narrow variants, while the standard configuration remained competitive in validation accuracy.

10.4.1 ResNet-18: Different Layer Widths

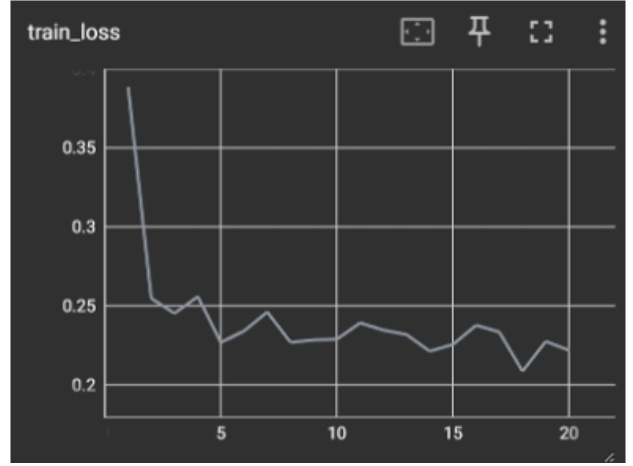
In addition to the ResNet34 width experiments, we evaluated whether changing the channel widths of our baseline Resnet-18 model leads to similar effects.

Table 9: ResNet-18 different layer widths

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
31	4	0.762431	0.522001	0.755814	0.618505	0.754811	0.754902
32	17	0.845304	0.453984	0.680233	0.754253	0.673883	0.674148



(a) Model 31 - Train Accuracy



(b) Model 31 - Train Loss

Compared configurations

- **Model 31 (narrow ResNet-18):** same training setup as the best baseline configuration (Model 1), but with reduced channel widths 32–64–128–256.
- **Model 32 (wide ResNet-18):** increased channel widths 96–192–384–768.

Observations The narrow configuration (Model 31) achieved the strongest overall results in this comparison, reaching **Test Acc** ≈ 0.756 and **macro/weighted F1** ≈ 0.755 . Importantly, this performance also **exceeds the earlier baseline Model 1** (Test Acc ≈ 0.737), indicating that reducing channel capacity improved generalization for ResNet-18 in our setting. In contrast, the wide configuration (Model 32) obtained higher validation accuracy (Val Acc ≈ 0.845) but performed substantially worse on the test set (Test Acc ≈ 0.680), suggesting overfitting or reduced robustness.

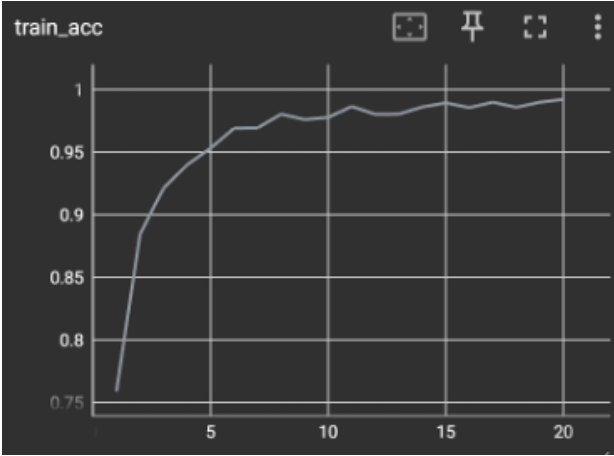
Conclusion For ResNet-18, decreasing the layer widths yielded a clear improvement in test performance and became the **preferred configuration for subsequent experiments**, while widening the network did not provide a generalization benefit despite stronger validation accuracy.

10.5 Effect of Removing Skip Connections (ResNet-18)

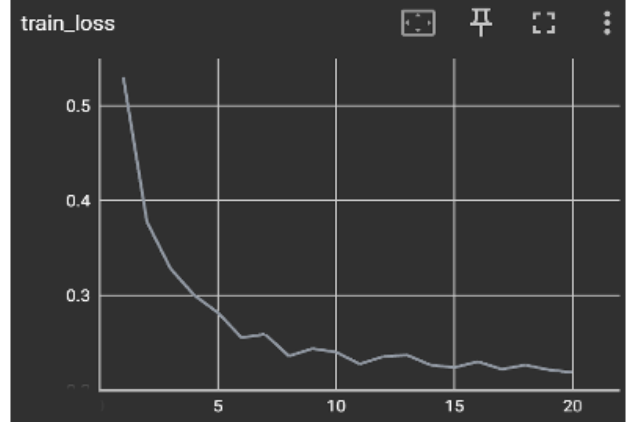
To quantify the contribution of residual learning, we evaluated a ResNet18 variant in which the **skip (identity) connections were removed**, while keeping the remaining training setup comparable. This experiment isolates the effect of the residual paths that normally allow each block to learn a residual function relative to its input.

Table 10: Resnet-18 without skip-connections

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
33	15	0.744015	0.600592	0.706395	0.718571	0.698693	0.698973



(a) Model 33 - Train Accuracy



(b) Model 33 - Train Loss

Configuration

- **Model 33:** ResNet-18-style architecture (Model 31) trained **without skip connections**.

Observations Removing skip connections led to a noticeable drop in validation performance (Val Acc ≈ 0.744) and produced **Test Acc** ≈ 0.706 with macro/weighted F1 ≈ 0.699 . Compared to the best ResNet18 configurations with residual connections, this indicates that the identity shortcuts are important for stable training and better generalization in this task. In practice, the residual formulation appears to provide a more effective optimization path and improves final classification performance.

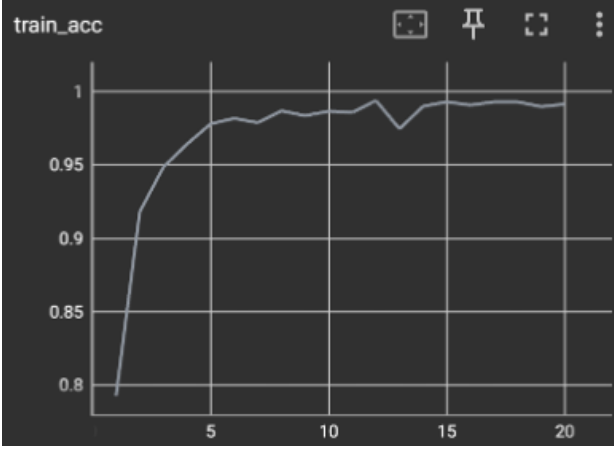
Conclusion The skip connections are a key component of the ResNet design in our setting. Eliminating them reduced both validation and test performance, so residual connections were retained in all subsequent model variants.

10.6 Batch Normalization Placement: BN After ReLU

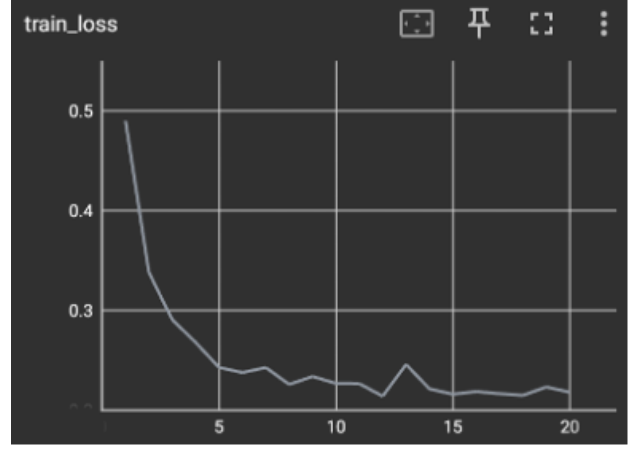
Next, we tested whether changing the ordering of operations inside the ResNet18 blocks affects performance. In the standard formulation used in our baseline, batch normalization is applied **before** the ReLU activation (Conv $\rightarrow$ BN $\rightarrow$ ReLU). In this experiment, we swapped the order to apply batch normalization **after** the activation (Conv $\rightarrow$ ReLU $\rightarrow$ BN) while keeping the remaining training setup unchanged.

Table 11: Resnet-18 batchnorm after Relu

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
34	8	0.832413	0.507476	0.672965	0.843473	0.671158	0.671300



(a) Model 34 - Train Accuracy



(b) Model 34 - Train Loss

Configuration

- **Model 34:** best ResNet-18 configuration (Model 31), but with **BatchNorm placed after ReLU** inside blocks(checkpoint selected by lowest validation loss).

Observations This change resulted in a reduction in generalization performance: although the validation accuracy remained relatively high (Val Acc ≈ 0.832), the test metrics were noticeably lower (Test Acc ≈ 0.673 , macro/weighted F1 ≈ 0.671) and test loss increased (approximately 0.843). This suggests that the modified ordering produced features that transferred less well to unseen data, even when the validation set did not indicate a large degradation.

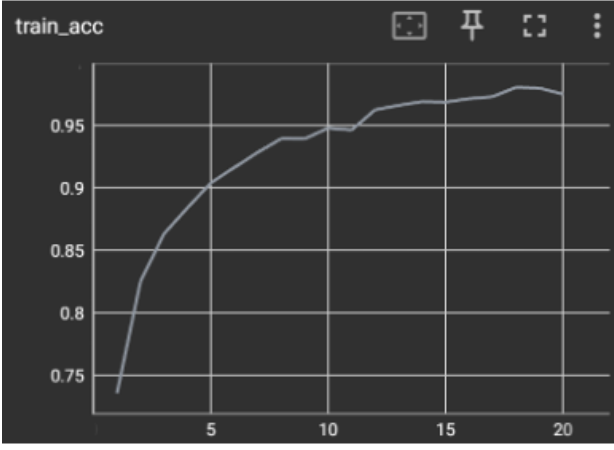
Conclusion Placing batch normalization after ReLU did not improve performance and led to weaker test results. Therefore, the standard ordering (Conv $\rightarrow$ BN $\rightarrow$ ReLU) was retained in subsequent experiments.

10.7 Activation Function: Sigmoid vs. ReLU

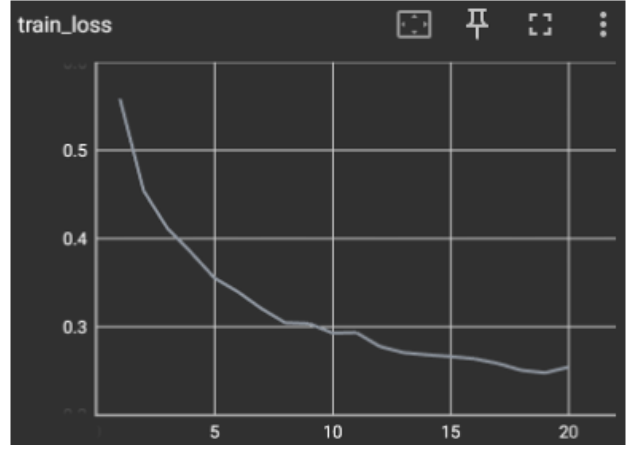
To evaluate whether the choice of non-linearity affects classification performance, we replaced the standard ReLU activations in the ResNet18 model with **sigmoid** activations and trained the network under several optimizer/regularization settings. The goal was to check if this architectural change improves generalization or training stability.

Table 12: ResNet-18 Sigmoid vs ReLU

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
35	10	0.755064	0.562922	0.629360	0.789762	0.628399	0.628509
36	3	0.589319	0.668555	0.553779	0.772019	0.540954	0.540507
37	17	0.736648	0.578005	0.713663	0.665245	0.709010	0.709224
38	2	0.677716	0.661848	0.649709	0.685617	0.644536	0.644785



(a) Model 37 - Train Accuracy



(b) Model 37 - Train Loss

Compared configurations (Table 12) All four models use the same underlying ResNet18 configuration as the best-performing setup (Model 31), but differ in optimization hyperparameters:

- **Model 35:** sigmoid instead of ReLU, best setup (Model 31).
- **Model 36:** sigmoid instead of ReLU, learning rate 1×10^{-3} .
- **Model 37:** sigmoid instead of ReLU, weight decay 1×10^{-4} .
- **Model 38:** sigmoid instead of ReLU, no weight decay ($\text{wd}=0$).

Observations Across all tested settings, replacing ReLU with sigmoid resulted in **weaker validation and test performance**. The best sigmoid-based run (Model 37) achieved Test Test Acc ≈ 0.714 and F1 ≈ 0.709 , which remains below the corresponding ReLU-based baseline (Model 31). Other sigmoid runs performed substantially worse (e.g., Model 36 with Test Acc ≈ 0.554). This indicates that sigmoid activations were not a good fit for this architecture and training regime, likely making optimization more difficult and reducing the quality of learned features.

Conclusion Using sigmoid instead of ReLU did not improve ResNet-18 performance in our experiments. ReLU activations were therefore retained in subsequent model variants

10.8 Resnet with pretrained

Following the methodology described earlier for augmentation on the not-pretrained model, the experiments were repeated using a currently best model. The same augmentation parameters (**AUG_RATIO** and the binary flags **USE_NOISE**, **USE_CHOP**, **USE_FAST**, **USE_SLOW**) were evaluated. The results are summarized in Table 13.

Based on these results, the configuration combining chopping, speeding up the recording, slowing down the recording (CFS) consistently achieved the best performance. Notably, including additive noise did not further improve the results for this model architecture and dataset.

In the second stage, the selected CFS configuration was further analyzed by varying the augmentation ratio (1.0, 0.8, 0.6, 0.4, 0.2). Comparative results are presented in Table.

The highest performance was obtained with **AUG_RATIO** = 1.0, indicating that applying the selected augmentations to the entire training dataset is more beneficial than mixing augmented and non-augmented samples. Reducing the augmentation ratio consistently led to lower performance, suggesting that the model benefits most from maximum exposure to the augmented variability.

Table 13: Best model: comparison of augmentation combinations (best checkpoint selected by validation criterion). Abbreviations: N — Noise, C — Chop, F — Fast, S — Slow.

Aug	Train Acc	Val Acc	Test Acc	Val F1 (Macro)	Val F1 (Weighted)	Test F1 (Macro)	Test F1 (Weighted)
0.0	0.9879	0.7274	0.7747	0.8015	0.8032	0.7712	0.7714
N	0.9866	0.7827	0.7427	0.8146	0.8161	0.7366	0.7368
C	0.9929	0.7201	0.7980	0.8702	0.8712	0.7969	0.7970
F	0.9882	0.7974	0.7500	0.8103	0.8129	0.7422	0.7424
S	0.9867	0.7514	0.7878	0.8102	0.8107	0.7862	0.7863
NC	0.9902	0.8011	0.7703	0.7989	0.7997	0.7683	0.7685
NF	0.9923	0.7293	0.7311	0.8356	0.8365	0.7267	0.7269
NS	0.9902	0.7643	0.7108	0.8338	0.8346	0.7049	0.7051
CF	0.9917	0.7919	0.7922	0.7868	0.7894	0.7888	0.7889
CS	0.9923	0.8840	0.7849	0.8834	0.8842	0.7826	0.7828
FS	0.9882	0.6888	0.7922	0.8703	0.8713	0.7913	0.7913
NCF	0.9925	0.8287	0.7442	0.8484	0.8493	0.7425	0.7426
NCS	0.9919	0.8122	0.8241	0.8103	0.8102	0.8241	0.8241
NFS	0.9913	0.8471	0.7267	0.8726	0.8732	0.7267	0.7267
CFS	0.9937	0.8214	0.8474	0.8500	0.8511	0.8469	0.8469
NCFS	0.9960	0.8398	0.7660	0.8421	0.8435	0.7618	0.7620

Table 14: Best model with CFS augmentation: effect of AUG_RATIO.

Aug Ratio	Train Acc	Val Acc	Test Acc	Val F1 (Macro)	Val F1 (Weighted)	Test F1 (Macro)	Test F1 (Weighted)
1.0	0.9937	0.8214	0.8474	0.8500	0.8511	0.8469	0.8469
0.8	0.9931	0.7366	0.7573	0.8662	0.8674	0.7568	0.7569
0.6	0.9885	0.8011	0.7863	0.8893	0.8898	0.7860	0.7860
0.4	0.9929	0.7808	0.7529	0.8248	0.8254	0.7527	0.7528
0.2	0.9854	0.7403	0.8110	0.8443	0.8455	0.8097	0.8098

For the pretrained ResNet18, the best-performing augmentation configuration was found to be **CFS**. Therefore, in the following experiments, we will use this combination with an augmentation ratio of 1.0, as it currently provides the most effective training setup.

11 Resnet pretrained - best result

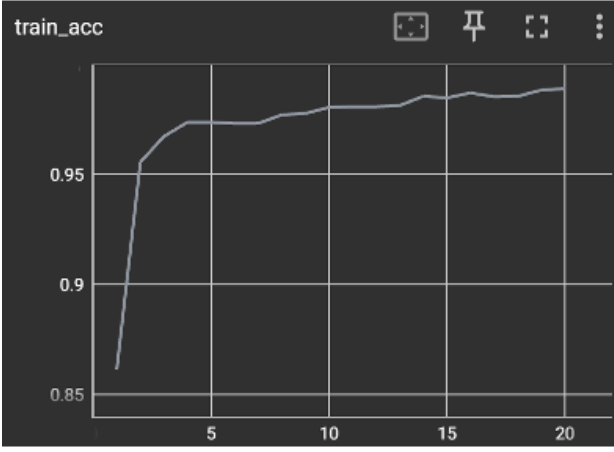
11.1 Impact of SpecAugment

After selecting the best-performing model configuration (the pretrained variant), we additionally evaluated spectrogram-level augmentation using a SpecAugment-style masking procedure applied directly to the mel-spectrogram inputs during training. Instead of modifying the waveform, this approach perturbs the time–frequency representation by randomly masking small contiguous regions, which increases input variability while keeping the overall structure of the spectrogram unchanged.

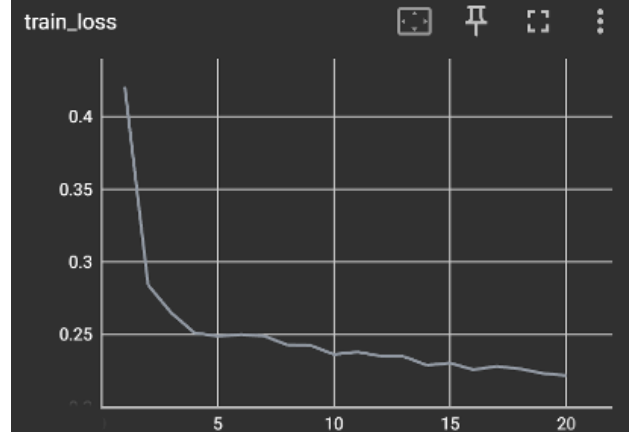
In our implementation, the augmentation consists of frequency masking and time masking. For each spectrogram, a small number of frequency bands and time intervals are selected at random and set to zero. This serves as an additional regularization mechanism when training the CNN on mel-spectrogram features.

Table 15: ResNet-18 with SpecAugment

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
1	4	0.758748	0.553731	0.725291	0.664309	0.721578	0.721666
2	19	0.854512	0.443329	0.728198	0.673166	0.718284	0.718481
3	17	0.813996	0.472369	0.760174	0.589597	0.756803	0.756970



(a) Model 3 - Train Accuracy



(b) Model 3 - Train Loss

Compared Configurations

- **Model 1:** time and frequency masking with a larger masking range (up to $\approx 10\%$ of frequency bins and time frames).
- **Model 2:** the same augmentation with a smaller masking range (up to $\approx 5\%$ of frequency bins and time frames).
- **Model 3:** Model 2 masking setup combined with **FC dropout** ($p = 0.3$).

Observations Reducing the masking strength (Model 2) substantially improved validation performance compared to Model 1 (Val Acc ≈ 0.855 vs. 0.759), indicating that a milder augmentation acted as a better regularizer than the stronger masking. However, the best **test performance** was achieved by **Model 3**, where mild masking was combined with dropout - it reached the highest test accuracy (≈ 0.760) and the strongest macro/weighted F1 (≈ 0.757), together with the lowest test loss among these three runs (≈ 0.590).

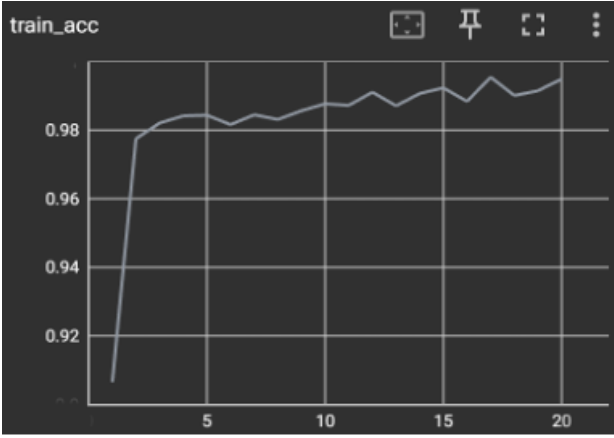
Conclusion Overall, the results suggest that **spectrogram augmentation is sensitive to its intensity**. Strong masking reduced performance, while milder masking improved validation results. The best generalization in this block was obtained when spectrogram masking was paired with additional regularization in the classifier head (dropout). Despite the potential regularization benefits, the SpecAugment-based training runs resulted in consistently lower performance than the baseline configuration without spectrogram augmentation.

11.2 Impact of Adding Dropout to the Best Pretrained Model

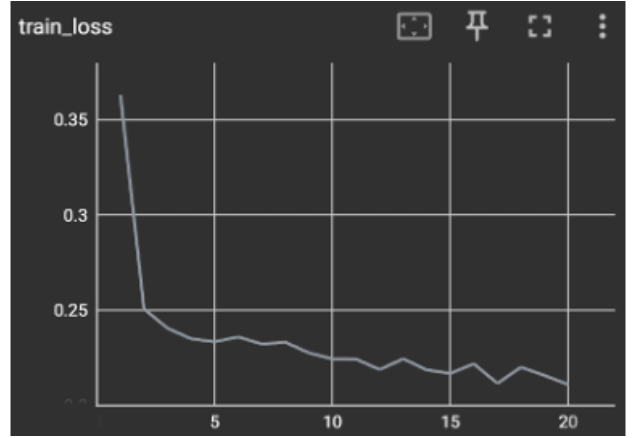
After selecting the strongest-performing baseline configuration, we evaluated whether adding dropout to the classifier improves generalization. In this experiment, the architecture and training setup were kept unchanged, and only a **fully connected (FC) dropout layer** was introduced with different probabilities p .

Table 16: Best pretrained ResNet-18 with FC dropout (best checkpoint selected by lowest validation loss).

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
4	7	0.867403	0.429311	0.784884	0.572140	0.782366	0.782502
5	11	0.852670	0.441822	0.765988	0.636716	0.761440	0.761631
6	11	0.834254	0.470984	0.704942	0.765459	0.701393	0.701583



(a) Model 4 - Train Accuracy



(b) Model 4 - Train Loss

Compared Configurations

- **Model 4:** baseline model + FC dropout ($p = 0.3$).
- **Model 5:** baseline model + FC dropout ($p = 0.4$).
- **Model 6:** baseline model + FC dropout ($p = 0.2$).

Observations Among the tested values, Model 4 ($p = 0.3$) produced the best overall results, achieving the highest validation accuracy (≈ 0.867) and the best test performance (Test Acc ≈ 0.785 , macro/weighted F1 ≈ 0.782) with the lowest test loss in this group (≈ 0.572). Increasing dropout (Model 5, $p = 0.4$) slightly reduced performance, while the lower dropout setting ($p = 0.2$) led to a clearer degradation in both validation and test metrics.

Conclusion Overall, introducing FC dropout did not provide a clear benefit over the baseline configuration. Therefore, dropout was not treated as an improvement in subsequent runs.

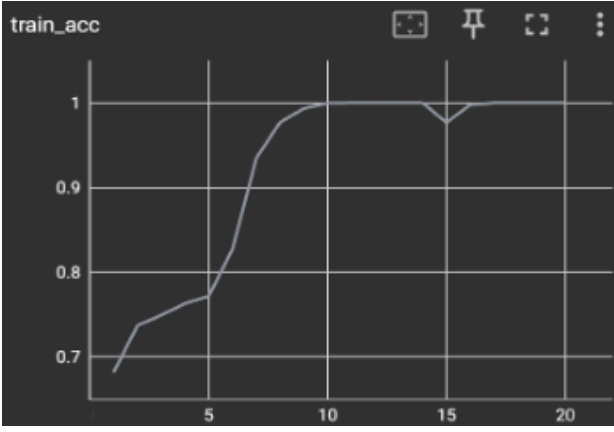
11.3 Pretraining, Transfer Learning, and Fine-Tuning

Finally, we investigated whether **fine-tuning** can further improve performance when using a **pre-trained backbone**. The best-performing model at this stage was pretrained by default, so the main question was how much of the network should be updated during training: (i) training only a new classifier on top of frozen features (transfer learning), or (ii) unfreezing part of the backbone and fine-tuning it.

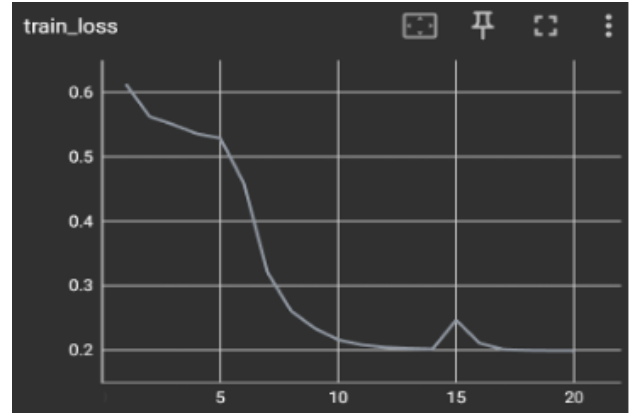
The results are summarized in Table 17.

Table 17: ResNet-18 and ResNet-34 pretraining, transfer learning, fine-tuning

Model	Epoch	Val Acc.	Val Loss	Test Acc.	Test Loss	Test F1 Macro	Test F1 Weighted
7	1	0.528545	0.738108	0.667151	0.636689	0.665312	0.665456
8	1	0.565378	0.736241	0.643895	0.641473	0.643346	0.643265
9	1	0.565378	0.736909	0.643895	0.641503	0.643346	0.643265
10	13	0.707182	0.648983	0.710756	0.626578	0.707277	0.707463
11	13	0.707182	0.632410	0.726744	0.616955	0.722999	0.723186
12	10	0.896869	0.387759	0.734012	0.693172	0.729689	0.729888
13	4	0.523020	0.719886	0.651163	0.644420	0.641667	0.642006
14	4	0.567219	0.719609	0.665698	0.619991	0.665355	0.665418



(a) Model 11 - Train Accuracy



(b) Model 11 - Train Loss

Compared Configurations

1. Frozen-backbone transfer learning (ResNet18)

- **Model 7:** frozen backbone - only the classifier trained.
- **Model 8:** frozen backbone with higher learning rate (1×10^{-3}).
- **Model 9:** frozen backbone with higher learning rate and lower weight decay ($\text{lr} = 1 \times 10^{-3}$, $\text{wd} = 1 \times 10^{-4}$).

2. Partial fine-tuning (ResNet18)

- **Model 10 (AdamW):** backbone frozen for epochs 1–4; from epoch 4 onward, **layer4** is unfrozen and trained with a smaller learning rate, while the classifier uses a larger learning rate ($\text{fc lr} = 1 \times 10^{-3}$, $\text{layer4 lr} = 1 \times 10^{-4}$).
- **Model 11 (Adam):** same partial fine-tuning schedule as **Model 10**, but using Adam instead of AdamW.

3. ResNet34 reference (pretraining vs frozen backbone)

- **Model 12:** pretrained ResNet34.
- **Model 13:** pretrained ResNet34 with frozen backbone (transfer learning).
- **Model 14:** frozen-backbone ResNet34 with adjusted optimization ($\text{lr} = 1 \times 10^{-3}$, $\text{wd} = 0$).

Observations Freezing the backbone and training only the classifier (Models 7–9) led to weak validation performance and did not match the results achieved by full pretrained training. Partial fine-tuning (Models 10–11) improved over the fully frozen setting, indicating that updating at least the final residual stage is beneficial. Among these, the Adam-based variant (Model 11) performed better on the test set than the AdamW variant (Model 10). However, none of the ResNet18 fine-tuning variants surpassed the fully pretrained reference.

For ResNet34, the pretrained configuration (Model 12) achieved clearly stronger validation metrics than the frozen-backbone variants (Models 13–14), again suggesting that allowing the backbone to adapt to the spectrogram domain is important.

Conclusion Overall, the results indicate that **freezing the entire backbone is too restrictive**, while **partial fine-tuning** provides a measurable improvement relative to frozen transfer learning. Nevertheless, the best performance in this set is still achieved by the **fully pretrained** model configuration.

12 Graphical User Interface

In the final stage of the project, the remaining task was to design a graphical user interface (GUI) for the trained model.

12.1 Implementation

In the final stage of this project, the last task to tackle was the graphical user interface for the model.

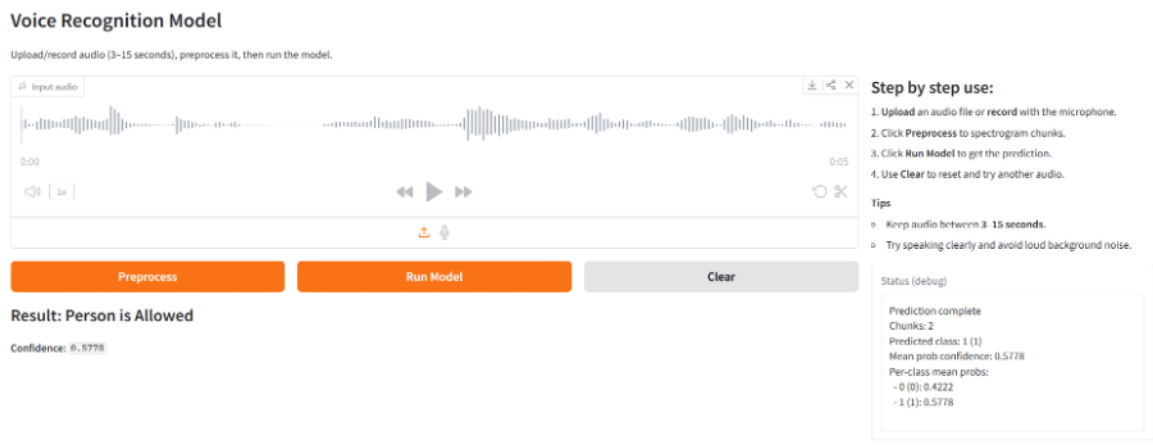


Figure 12: This is the finished GUI result

The interface was created using the Gradio framework used as a python library in a Jupyter Notebook environment.

The interface allows the user to either upload an audio file or record a short speech sample directly through the browser.

Since this is a model to essentially recognize someone for some access validation, we limited the audio to be 3–15 seconds long.

Behind the GUI once the audio file is inputted, it is preprocessed using the same steps as it was done for our data in the dataset, that is (in short summary):

- silence removal
- normalization
- segmentation into 3-second chunks
- conversion to log-Mel spectrograms
- formatting into the input shape expected by the neural network

Next the user can run the model and a prediction is displayed with a result indicating whether the recording belongs to Class 0 or Class 1.

Additionally the confidence score of the decision is displayed.

On the side a user can find simple instructions and tips for using the GUI, as well as a status panel presenting any technical details during each step.